

RGraphSpace: Rendering graphs as coherent spatial objects in ggplot2

Flávio Gabriel Carazza-Kessler^{1*}, Jonathan André Back^{1*}, Lana Bazan Peters Querne^{1*}, Victor Henrique Apolonio dos Santos¹, and Mauro Antonio Alves Castro¹

¹ Bioinformatics and Systems Biology Laboratory, Federal University of Paraná, Curitiba, PR, 81520-260, Brazil  Corresponding author * These authors contributed equally.

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Nikoleta Glynatsi](#) 

Reviewers:

- [@schochastics](#)
- [@christopherkenny](#)

Submitted: 01 August 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

A graph describes a network structure of nodes connected by edges, and is used across many areas of science to represent relationships between entities. When the relationships themselves are spatial, node positions carry physical meaning, and the graph must be further represented within an external frame of reference. In R, *igraph* (Csardi & Nepusz, 2006) provides powerful tools for network analysis, and the *ggplot2* ecosystem (Wickham, 2016) for visualization, drawing nodes and edges as separate layers. Integrating the two remains challenging, however, because the geometry of these layers is not fully coordinated at rendering time, a consequence of *ggplot2*'s input model, in which each plot layer consumes a single rectangular table and is resolved independently. This design works remarkably well for most applications but is restrictive for graph structures with mutually dependent components. Because layers do not share transformed states, the graph is not rendered as a single coherent object. Here we introduce *RGraphSpace*, a lightweight interface that integrates *igraph* with *ggplot2* graphics within a normalized coordinate space. It synchronizes node and edge layers, jointly resolving their geometry under standard aesthetic mappings. This coherence lets the graph be anchored to external reference frames, where it becomes a spatial object embedded in a wider context. *RGraphSpace* is available from CRAN, with comprehensive tutorials provided on its documentation site (<https://sysbiolab.github.io/RGraphSpace>).

Statement of need

Graph handling in R rests on two mature foundations: *igraph* (Csardi & Nepusz, 2006) for network analysis and *ggplot2* (Wickham, 2016) for layered visualization. The *ggraph* package (Pedersen, 2025a) bridges these, exposing relational data to the grammar of graphics. Graph data manipulation has similarly matured through *tidygraph* (Pedersen, 2025b), which represents graphs as tidy node and edge tables.

Yet rendering a graph as a spatial object requires coordination across its components. Because *ggplot2* resolves each layer independently, node and edge layers are not fully synchronized. Positional coordinates are shared, but other aesthetics are resolved per layer, so edges have no access to the rendered sizes of the nodes.

RGraphSpace addresses this by mapping an *igraph* object into a normalized coordinate space in which nodes, edges, and their associated elements are resolved together. With node and edge geometries synchronized at rendering time, edges are drawn with respect to the final extent of the nodes they connect, and this correspondence is maintained even when node attributes are rescaled through the *ggplot2* grammar.

This normalized space serves a second purpose, placing the graph within a broader spatial context, aligned to external reference frames such as microscopy images. Graph-image alignment requires consistent coordinate conventions; a mismatch in axis orientation, for example a top-left versus bottom-left origin, leaves the nodes reflected relative to the image (Figure 1A). Moreover, because each pixel cell occupies a finite square area, a half-pixel offset is needed to prevent node positions from drifting toward the borders (Figure 1B). This discrepancy may seem negligible but can matter when nodes represent measurements anchored to specific spatial features. Reconciling these conventions lets the graph be rendered as a coherent spatial object rather than an isolated diagram.

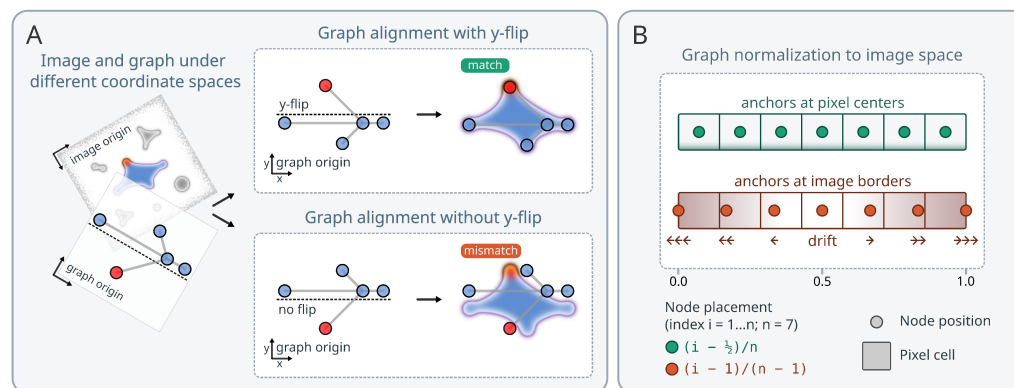


Figure 1: Spatial alignment. (A) Aligning a toy graph to a background image. Left: node coordinates are assumed to be pixel indices of the reference image, but rendering both in *ggplot2* requires a shared, normalized coordinate space. Top right: with y-flip, the two are rendered in correspondence. Bottom right: without it, the graph reflects relative to the image. The red node breaks symmetry and highlights orientation. (B) Two normalization schemes, where i denotes a pixel or node index along the axis. Top: *anchors at pixel centers* use $(i - \frac{1}{2})/n$, introducing a half-pixel offset. Bottom: *anchors at image borders* use $(i - 1)/(n - 1)$, mapping positions across the full normalized interval $[0, 1]$. The two coincide mid-axis but drift apart toward the ends. *RGraphSpace* applies the y-flip and pixel-center anchoring by default.

State of the field

The Grammar of Graphics (Wilkinson, 2005) constructs statistical graphics by decomposing a plot into independent, composable elements: data, aesthetic mappings, geometric objects, scales, coordinate systems, and statistical transformations. A graphic is then built declaratively by combining these components. In R, the *ggplot2* package implements a layered form of this grammar (Wickham, 2010), assembling graphics by adding successive layers to a shared coordinate system, and has become the dominant visualization framework in the R ecosystem.

Central to this design is how *ggplot2* consumes data: each layer is bound to a single rectangular table and resolved independently, applying its own transformations, scales, and position adjustments before being drawn (Wickham, 2016). This independence is a deliberate strength, since layers can be freely combined, reordered, and reused, and it pairs naturally with the tidy data convention (Wickham, 2014), in which each variable is a column and each observation a row.

This design has allowed *ggplot2* to accommodate data structures beyond simple flat tables. The *sf* package (Pebesma, 2018), for example, represents spatial vector data as list-columns and integrates with *ggplot2* through a dedicated *geom_sf* layer, extending the grammar to handle spatial geometry while preserving its declarative interface. Graph data has received similar attention. The *ggraph* package (Pedersen, 2025a) extends *ggplot2* with an extensive family of node and edge geometries, bringing graph layouts into the grammar, while other tools

69 such as *ggnetwork* (Briatte, 2026) and *GGally* (Schloerke et al., 2025) provide comparable
70 functionality for plotting network data. These packages establish that the node and edge tables
71 of a graph can be expressed through aesthetic mappings and rendered as separate *ggplot2*
72 layers. They do not, however, synchronize node and edge geometry during rendering.

73 Alongside these visualization tools, a parallel line of work has focused on making complex
74 data structures accessible to the tidyverse (Wickham et al., 2019), a collection of R packages
75 sharing a common design for data manipulation. The *tidygraph* package (Pedersen, 2025b)
76 represents a graph as a pair of tidy node and edge tables, allowing graph manipulation through
77 familiar tidyverse verbs while preserving its relational structure. This tidy philosophy extends
78 to multi-component containers common in computational biology (Hutchison et al., 2024),
79 including *Seurat* (Hao et al., 2024) and *SpatialExperiment* (Righelli et al., 2022), which
80 bundle components such as assays, metadata, reductions, spatial coordinates, and graphs
81 within a single object. *RGraphSpace* builds on this well-supported foundation, integrating with
82 *tidygraph* to contribute the rendering step.

83 Research impact statement

84 *RGraphSpace* provides the spatial foundation for the *PathwaySpace* package (Sysbiolab Team,
85 2026), which projects network signals onto a normalized coordinate space, transforming discrete
86 vertex signals into continuous surfaces over the graph topology. This integration has supported
87 published analyses in systems biology (Ellrott et al., 2025; Tercan et al., 2025), demonstrating
88 *RGraphSpace*'s utility for downstream spatial analysis tools.

89 Software design

90 *RGraphSpace* is built around the *GraphSpace* S4 class, which encapsulates a graph together
91 with the components required for coherent rendering: node and edge tables, feature data,
92 source and render-ready images, and metadata (Figure 2A). Object validity enforces a shared
93 node identity across the graph vertices, node table, and feature rows. Rather than embedding
94 node-associated features as node attributes, *GraphSpace* stores them in `@fdata`, a sparse-matrix
95 slot aligned to nodes but independent of the node table. When a feature is mapped to an
96 aesthetic, only the requested feature is retrieved for plotting, so graphs carrying thousands of
97 features are never expanded into dense node tables.

98 Graphs can be supplied as either *igraph* or *tidygraph* objects through a common interface
99 (Figure 2B). *RGraphSpace* also works with *ggraph*, accepting its layouts as input and providing
100 geometries that can be used within *ggraph* plots. Coercion methods extend this interface to
101 selected multi-component containers, loading their node-associated features into the `@fdata`
102 slot.

103 The `normalizeGraphSpace()` function scales node coordinates into a unit square aligned to
104 a reference frame. By default, this frame is the graph's own extent, centered within the
105 square; when a background image is available, the image space is used instead. In this case,
106 node coordinates are assumed to be pixel indices of the image matrix. The orientation and
107 pixel-anchoring corrections are depicted in Figure 1. The `normalizeGraphSpace()` function
108 also exposes arguments for graph-image alignment operations, demonstrated in the online
109 tutorials.

110 A *GraphSpace* supplied to `ggplot()` produces a subclassed plot that verifies, through a shared
111 `@uuid`, that the node and edge layers originate from the same graph before coordinating them.
112 *RGraphSpace* then intercepts the build pipeline after the node layer has been fully processed,
113 and the resulting rendered sizes are passed to the edge layer, allowing edge construction to
114 use the final node representation. This synchronization is available through three geometries:
115 `geom_nodesspace()`, `geom_edgespace()`, and the convenience wrapper `geom_graphspace()`.

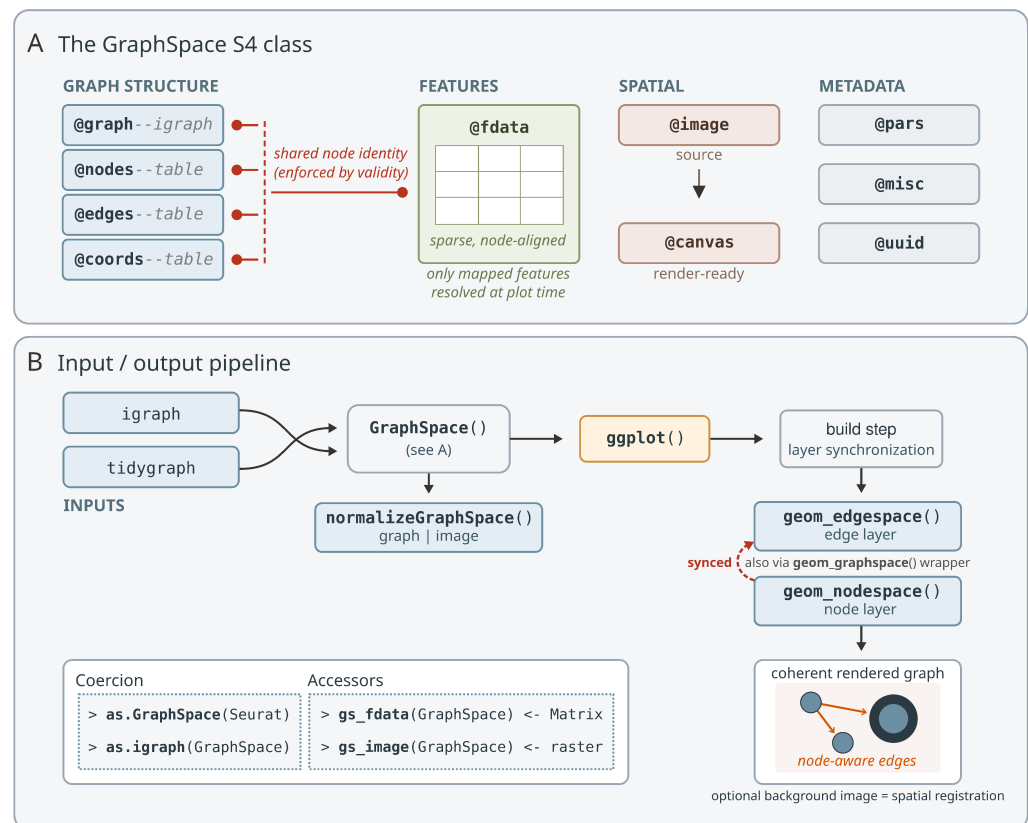


Figure 2: Architecture of *RGraphSpace*. (A) The *GraphSpace* S4 class stores a graph, its node and edge tables, feature data, source and render-ready images, and metadata. Node identity is enforced by `setValidity`, and `@coords` preserves the original coordinates for non-destructive transformations. (B) Graph inputs (`igraph` or `tidygraph`) are used to construct a *GraphSpace*; selected non-graph objects are handled through coercion and accessors (inset). Coordinates are normalized to a unit square, centered on the graph extent or aligned to a background image. At the *ggplot2* build step, the node and edge layers are synchronized and rendered over an optional background image.

Availability and documentation

RGraphSpace is available from CRAN (<https://cran.r-project.org/package=RGraphSpace>) and the development version is hosted on GitHub (<https://github.com/sysbiolab/RGraphSpace>). Documentation is available at <https://sysbiolab.github.io/RGraphSpace> and includes comprehensive tutorials covering the full workflow, illustrated here by graph construction from an `igraph` object (Figure 3A), a graph rendered as a spatial object over a background image (Figure 3B), and application to spatial feature data (Figure 3C).

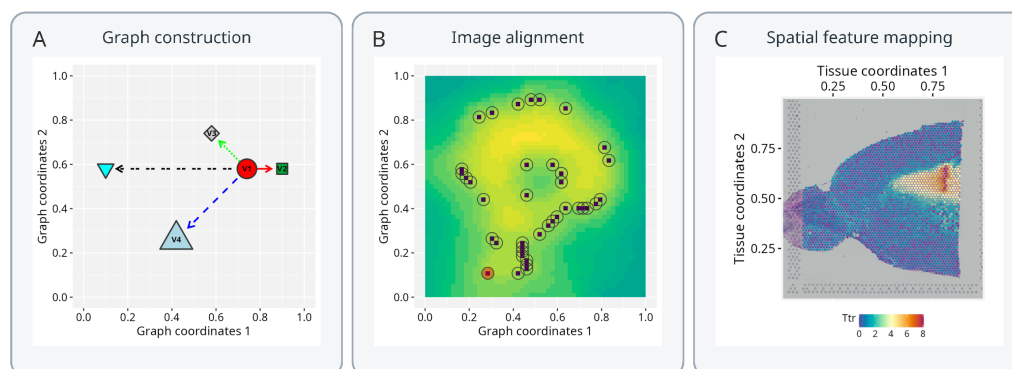


Figure 3: Selected tutorials available at <https://sysbiolab.github.io/RGraphSpace>. (A) Graph construction: a graph rendered from an *igraph* object using *RGraphSpace* geometries. (B) Image alignment: a graph rendered as a spatial object over a background image. (C) Spatial feature mapping: graph and feature data rendered together, illustrated with spatial transcriptomics data.

AI usage disclosure

During the preparation of this work, the authors used ChatGPT (OpenAI) and Claude Code (Anthropic) to improve text readability and to audit code while using RStudio Desktop (<https://posit.co/>). The authors carefully reviewed and edited the content as needed after using these tools and assume full responsibility for the published content.

Acknowledgements

This work was funded by CNPq (440412/2022-6 and 307144/2025-9), CAPES (Finance Code 001), and Fundação Araucária (NAPI Bioinformática).

References

- Briatte, F. (2026). *Ggnetwork: Geometries to plot networks with 'ggplot2'*. <https://doi.org/10.32614/CRAN.package.ggnetwork>
- Csardi, G., & Nepusz, T. (2006). The igraph software package for complex network research. *InterJournal, Complex Systems*, 1695. <https://igraph.org/>
- Ellrott, K., Wong, C. K., Yau, C., Castro, M. A. A., & others. (2025). Classification of non-TCGA cancer samples to TCGA molecular subtypes using compact feature sets. *Cancer Cell*, 43(2), 195–212.e11. <https://doi.org/10.1016/j.ccell.2024.12.002>
- Hao, Y., Stuart, T., Kowalski, M. H., Choudhary, S., Hoffman, P., Hartman, A., Srivastava, A., Molla, G., Madad, S., Fernandez-Granda, C., & Satija, R. (2024). Dictionary learning for integrative, multimodal and scalable single-cell analysis. *Nature Biotechnology*, 42(2), 293–304. <https://doi.org/10.1038/s41587-023-01767-y>
- Hutchison, W. J., Keyes, T. J., & The tidyomics Consortium. (2024). The tidyomics ecosystem: Enhancing omic data analyses. *Nature Methods*, 21, 1166–1170. <https://doi.org/10.1038/s41592-024-02299-2>
- Pebesma, E. (2018). Simple features for R: Standardized support for spatial vector data. *The R Journal*, 10(1), 439–446. <https://doi.org/10.32614/RJ-2018-009>
- Pedersen, T. L. (2025a). *Ggraph: An implementation of grammar of graphics for graphs and networks*. <https://doi.org/10.32614/CRAN.package.ggraph>

- 150 Pedersen, T. L. (2025b). *Tidygraph: A tidy API for graph manipulation*. <https://doi.org/10.32614/CRAN.package.tidygraph>
- 151
- 152 Righelli, D., Weber, L. M., Crowell, H. L., Pardo, B., Collado-Torres, L., Ghazanfar, S.,
153 Lun, A. T. L., Hicks, S. C., & Risso, D. (2022). SpatialExperiment: Infrastructure for
154 spatially-resolved transcriptomics data in r using bioconductor. *Bioinformatics*, 38(11),
155 3128–3131. <https://doi.org/10.1093/bioinformatics/btac299>
- 156 Schloerke, B., Cook, D., Larmarange, J., Briatte, F., Marbach, M., Thoen, E., Elberg, A.,
157 & Crowley, J. (2025). *GGally: Extension to 'ggplot2'*. <https://doi.org/10.32614/CRAN.package.GGally>
- 158
- 159 Sysbiolab Team. (2026). *PathwaySpace: Spatial projection of network signals along geodesic*
160 *paths*. <https://doi.org/10.32614/CRAN.package.PathwaySpace>
- 161 Tercan, B., Apolonio, V. H., Chagas, V. S., Wong, C. K., & others. (2025). Protocol
162 for assessing distances in pathway space for classifier feature sets from machine learning
163 methods. *STAR Protocols*, 6(2), 103681. <https://doi.org/10.1016/j.xpro.2025.103681>
- 164 Wickham, H. (2010). A layered grammar of graphics. *Journal of Computational and Graphical*
165 *Statistics*, 19(1), 3–28. <https://doi.org/10.1198/jcgs.2009.07098>
- 166 Wickham, H. (2014). Tidy data. *Journal of Statistical Software*, 59(10), 1–23. <https://doi.org/10.18637/jss.v059.i10>
- 167
- 168 Wickham, H. (2016). *ggplot2: Elegant graphics for data analysis*. Springer-Verlag New York.
169 <https://doi.org/10.1007/978-3-319-24277-4>
- 170 Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Golemund,
171 G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M.,
172 Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., ... Yutani, H. (2019). Welcome
173 to the tidyverse. *Journal of Open Source Software*, 4(43), 1686. <https://doi.org/10.21105/joss.01686>
- 174
- 175 Wilkinson, L. (2005). *The grammar of graphics* (2nd ed.). Springer-Verlag. <https://doi.org/10.1007/0-387-28695-0>
- 176